

# Brazo Robótico con Arduino UNO Q

Documentación Técnica del Proyecto

Sesión 2026-05-06

Brazo Robótico con Arduino UNO Q

Descripción General

Hardware

Cableado

Mapeo de Servos (pendiente de confirmación)

Arquitectura Software

Flujo de Datos

Cinemática Inversa (IK)

Stack Tecnológico del Proyecto

Servicios en Qualcomm Linux

Instalación y Despliegue

Errores y Soluciones

Estado del Proyecto

Árbol del Proyecto

# Brazo Robótico con Arduino UNO Q

## Descripción General

Mano robótica con 5 dedos accionados por servomotores Tower Pro MG996R, controlada por una arquitectura de doble placa: Arduino UNO Q + Arduino Mega 2560. El sistema copia en tiempo real los movimientos de una mano humana detectados mediante cámara usando MediaPipe.js.

## Hardware

| Componente       | Modelo                   | Función                                 |
|------------------|--------------------------|-----------------------------------------|
| Placa principal  | Arduino UNO Q (ABX00162) | Control central, WiFi, servidor web, IA |
| Placa secundaria | Arduino Mega 2560        | Control directo de servos (PWM)         |
| Servos ×5        | Tower Pro MG996R         | Accionamiento de dedos                  |
| Shield           | MEGA Sensor Shield V2.0  | Distribución de señales y alimentación  |
| Fuente           | Switching 5V / 10A+      | Alimentación de servos y Mega           |

## Arduino UNO Q — Especificaciones

| Característica | Valor                                             |
|----------------|---------------------------------------------------|
| MPU            | Qualcomm Dragonwing QRB2210 (4×Cortex-A53 @ 2GHz) |
| MCU            | STM32U585 (Cortex-M33 @ 160-MHz)                  |
| RAM            | 2 GB                                              |
| SO             | Debian Linux + Zephyr OS                          |
| PWM pins       | 6 (D3, D5, D6, D9, D10, D11)                      |
| I/O voltaje    |                                                   |

| Característica      | Valor                                                          |
|---------------------|----------------------------------------------------------------|
|                     | 3.3V lógica, 5V tolerante (excepto A0/A1)                      |
| <b>Conectividad</b> | WiFi 5 dual-band, Bluetooth 5.1, USB-C                         |
| <b>LED Matrix</b>   | 12×8 LEDs rojos                                                |
| <b>Software</b>     | Arduino App Lab (ambos chips) /<br>Arduino IDE 2.0+ (solo MCU) |

## MG996R — Especificaciones

| Característica | Valor                            |
|----------------|----------------------------------|
| Torque         | 9.4 kg·cm (4.8V) / 11 kg·cm (6V) |
| Velocidad      | 0.17 seg/60° (4.8V)              |
| Ángulo         | 0° a 180°                        |
| Señal PWM      | 50 Hz, pulso 1–2 ms              |
| Voltaje        | 4.8V – 7.2V                      |
| Corriente      | ≈2.5A por servo bajo carga       |

## Cableado

UNO Q GND — Mega GND  
 UNO Q D1 (TX) — Mega D15 (RX3)  
 UNO Q D0 (RX) — Mega D14 (TX3)

### ADVERTENCIAS:

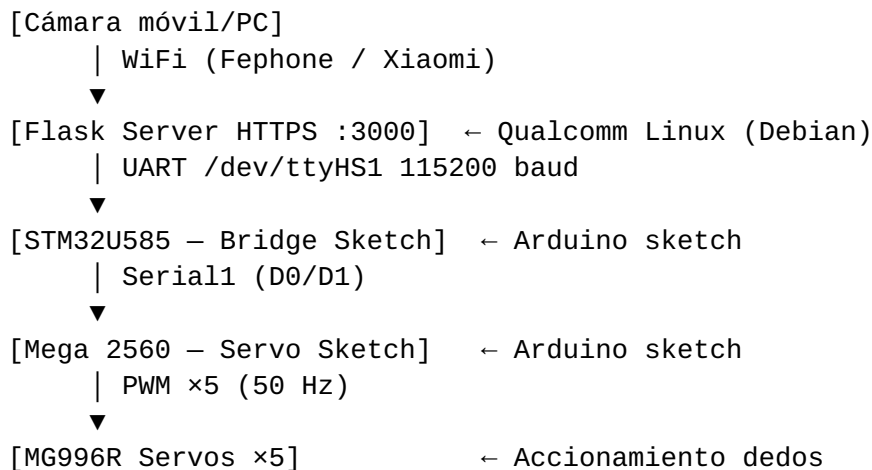
- Los servos NUNCA se alimentan desde el pin 5V del UNO Q. Usar fuente externa.
- El GND de la fuente externa debe estar conectado al GND común.
- La señal PWM del UNO Q es 3.3V; los MG996R la aceptan (umbral TTL ≈2.5V).

## Mapeo de Servos (pendiente de confirmación)

| Servo | Dedo   | Pin Mega | Pin UNO Q PWM |
|-------|--------|----------|---------------|
| 0     | Pulgar | ?        | D3            |

| Servo | Dedo    | Pin Mega | Pin UNO Q PWM |
|-------|---------|----------|---------------|
| 1     | Índice  | ?        | D5            |
| 2     | Corazón | ?        | D6            |
| 3     | Anular  | ?        | D9            |
| 4     | Meñique | ?        | D10           |

## Arquitectura Software



## Protocolo Serial UNO Q → Mega

- **Baudrate:** 115200
- **Formato:** M<servo\_idx> <angle>\n
- **Ejemplo:** M0 90\n → servo 0 a 90 grados

## Comunicación Qualcomm ↔ STM32 (descubierto)

| Componente     | Detalle                                                             |
|----------------|---------------------------------------------------------------------|
| UART física    | /dev/ttyHS1 (Qualcomm) = LPUART1 (STM32U585) @ 115200 baud          |
| Arduino Serial | Serial = Router Bridge (RPC),<br>Serial2 = acceso directo a LPUART1 |
| Router         | arduino-router.service, socket<br>Unix /var/run/arduino-router.sock |
| GPIO control   | 37=Reset STM32, 70=Ready,<br>38=Toggle shutdown                     |

| Componente | Detalle                                               |
|------------|-------------------------------------------------------|
| Fix reset  | Drop-in systemd vaciando ExecS-topPost y ExecStartPre |

## LED Matrix — Indicador de dedos

La matriz LED 12×8 del UNO Q se controla con **ArduinoGraphics** (fuente Font\_5x7 real, sin mapeo de bits manual):

| Dedos levanta-dos   | Display      | Implementación                                   |
|---------------------|--------------|--------------------------------------------------|
| 0 (tracking / puño) | 😊 Smiley     | <code>22 × matrix.point(x,y)</code>              |
| 1 a 5               | Número (1-5) | <code>matrix.text("5", 3, 0)</code> con Font_5x7 |

El sketch STM32 parsea los comandos `M<x> <angle>`, cuenta dedos con `servo_angle[i] < umbral`, y refresca la matriz.

### Umbrales por dedo (calibrados para el hardware real):

| Dedo                             | Umbral “abierto” |
|----------------------------------|------------------|
| Pulgar (servo 0)                 | < 145°           |
| Índice, Corazón, Anular, Meñique | < 50°            |

Anti-parpadeo: 400ms de histéresis antes de cambiar el display.

## Flujo de Datos

1. Cámara captura mano humana (móvil, PC o webcam)
2. MediaPipe.js detecta 21 landmarks (x, y, z) en el navegador
3. Filtro EMA ( $\alpha=0.3$ ) suaviza los landmarks para eliminar jitter
4. WebSocket envía coordenadas al servidor Flask
5. IK Engine convierte landmarks → ángulos de 5 servos
6. Comandos viajan: Flask → /dev/ttyHS1 → STM32 Serial2 → Serial1 (D0/D1) → Mega Serial3
7. Mega interpola suavemente (rampa 1°/12ms) los ángulos actuales hacia los target
8. Servos MG996R replican la posición de los dedos

## Control de Movimiento Suave

| Capa          | Ubicación         | Función                            |
|---------------|-------------------|------------------------------------|
| EMA           | Navegador (JS)    | Suaviza landmarks, elimina temblor |
| Deadband      | Servidor (Python) | Ignora cambios < 5°                |
| Interpolación | Mega (Arduino)    | Rampa 1°/12ms hacia target         |

## Cinemática Inversa (IK)

### Landmarks de MediaPipe

|              |               |                |     |
|--------------|---------------|----------------|-----|
| 0: WRIST     | 5: INDEX_MCP  | 9: MIDDLE_MCP  | 13: |
| RING_MCP     | 17: PINKY_MCP |                |     |
| 1: THUMB_CMC | 6: INDEX_PIP  | 10: MIDDLE_PIP | 14: |
| RING_PIP     | 18: PINKY_PIP |                |     |
| 2: THUMB_MCP | 7: INDEX_DIP  | 11: MIDDLE_DIP | 15: |
| RING_DIP     | 19: PINKY_DIP |                |     |
| 3: THUMB_IP  | 8: INDEX_TIP  | 12: MIDDLE_TIP | 16: |
| RING_TIP     | 20: PINKY_TIP |                |     |
| 4: THUMB_TIP |               |                |     |

### Algoritmo (v2 — mejorado con investigación de repos GitHub)

**Pulgar:** Comparación del eje X (tip vs IP) — enfoque usado por el 90% de los repositorios. Si  $\text{thumb\_tip.x} < \text{thumb\_ip.x} \rightarrow$  cerrado ( $\approx 170^\circ$ ), si no  $\rightarrow$  abierto ( $\approx 30^\circ$ ).

**4 dedos largos:** Ángulo 3D en PIP entre vectores MCP $\rightarrow$ PIP y PIP $\rightarrow$ TIP.  
 $\text{flexion\_3d} = \text{angle\_between\_3d}(\text{TIP-PIP}, \text{MCP-PIP})$   
 $\text{raw} = (180 - \text{flexion\_3d}) \times 0.85$   
 $\text{servo} = \text{lerp}(\text{raw}, \text{finger\_calib}[\text{open}], \text{finger\_calib}[\text{closed}])$

### Parámetros de estabilidad

| Parámetro       | Valor | Origen              |
|-----------------|-------|---------------------|
| EMA $\alpha$    | 0.25  | Consenso HCI        |
| Dead zone dedos | 8°    | logandul/Robot-Hand |

| Parámetro        | Valor    | Origen          |
|------------------|----------|-----------------|
| Dead zone pulgar | 12°      | (mayor rango)   |
| Rate limiting    | 5°/frame | Múltiples repos |
| Estabilidad      | 3 frames | Ihebzaouali     |
| Sensibilidad     | 85%      | Ajuste empírico |
| Send interval    | 30ms     | Sweet spot      |

### Calibración por dedo

| Dedo    | Open (servo) | Closed (servo) |
|---------|--------------|----------------|
| Pulgar  | 40°          | 160°           |
| Índice  | 0°           | 170°           |
| Corazón | 0°           | 170°           |
| Anular  | 0°           | 170°           |
| Meñique | 0°           | 170°           |

### Mejora: buffer estático (skill Embedded Firmware Engineer)

Siguiendo las reglas de la skill **Embedded Firmware Engineer** del Agency Agents:

| Regla                                       | Aplicación                                        |
|---------------------------------------------|---------------------------------------------------|
| “Never use dynamic allocation after init”   | String cmd_value → char<br>cmd_buffer[8] estático |
| “Always check bounds”                       | cmd_pos < 7 antes de escribir en<br>buffer        |
| “Avoid global mutable state unsynchronized” | servo_angle[] accedido solo en<br>loop (no ISR)   |

Skills instaladas en el proyecto (.opencode/agents/): 70, incluyendo Embedded Firmware Engineer, Code Reviewer, Technical Writer, Software Architect, Document Generator.

# Stack Tecnológico del Proyecto

| Capa         | Tecnología                                             | Archivo                            |
|--------------|--------------------------------------------------------|------------------------------------|
| Frontend     | HTML5 + MediaPipe.js + Three.js + Chart.js + WebSocket | qualcomm/static/index.html         |
| Backend      | Python3 + Flask + Flask-Sock                           | qualcomm/server.py                 |
| IK Engine    | Python3 + math (vectores 3D)                           | qualcomm/server.py (integrado)     |
| STM32 Bridge | Arduino C++ + ArduinoGraphics + LED Matrix             | stm32_sketch/bridge/bridge.ino     |
| Mega Servos  | Arduino C++ + Servo.h                                  | mega_sketch/servos/mega_servos.ino |
| WiFi AP      | hostapd + dnsmasq                                      | qualcomm/wifi_ap_setup.sh          |
| PDF          | Pandoc + WeasyPrint                                    | docs/                              |

## Funcionalidades del frontend

| Funcionalidad         | Tecnología                 | Estado |
|-----------------------|----------------------------|--------|
| Tracking de mano      | MediaPipe.js WASM          | ✓      |
| Indicadores de servos | HTML + CSS Grid            | ✓      |
| Mano 3D               | Three.js (carga asíncrona) | ✓      |
| Gráfica de ángulos    | Chart.js (carga asíncrona) | ✓      |
| Grabación             | JSON → descarga            | ✓      |
| Reproducción          | Carga JSON o memoria       | ✓      |
| Glassmorphism         | CSS backdrop-filter        | ✓      |

## Dependencias Python

- Flask  $\geq$  3.0
- Flask-Sock  $\geq$  0.7
- pyserial  $\geq$  3.5
- numpy  $\geq$  1.24
- waitress  $\geq$  3.0 (fallback WSGI)



# Servicios en Qualcomm Linux

| Servicio                      | Función                                          |
|-------------------------------|--------------------------------------------------|
| arduino-router.service        | Comunicación con STM32U585<br>vía /dev/ttyHS1    |
| arduino-router-serial.service | Proxy SOCAT: ttyGS0 → TCP:7500<br>(monitor)      |
| arduino-app-cli.service       | CLI para App Lab                                 |
| robot-hand.service            | <b>Servidor Flask</b> (auto-arranque<br>al boot) |

## Comunicación Qualcomm ↔ STM32

- **UART física:** /dev/ttyHS1 @ 115200 baud
- **Gestor:** arduino-router → socket Unix /var/run/arduino-router.sock
- **STM32-usb:** driver registrado (major 237)
- **GPIO control:** gpioset -c /dev/gpiochip1 37=0 (reset STM32), 70=1 (ready)

## Instalación y Despliegue

### En Qualcomm Linux

```
# Dependencias sistema
sudo apt-get update
sudo apt-get install -y python3-pip hostapd dnsmasq

# Dependencias Python
pip3 install --break-system-packages flask flask-sock pyserial
numpy waitress

# WiFi AP (alternativo, para modo standalone)
sudo tee /etc/hostapd/hostapd.conf << EOF
interface=wlan0
driver=nl80211
ssid=RobotHand
hw_mode=g
channel=6
wpa=2
wpa_passphrase=robot2026
wpa_key_mgmt=WPA-PSK
EOF
```

```
# Copiar archivos del proyecto
scp -r qualcomm/ arduino@<IP>:~/

# Iniciar servidor (manual)
cd ~/qualcomm && python3 server.py &

# Servicio systemd (auto-arranque)
mkdir -p ~/.config/systemd/user/
# Crear ~/.config/systemd/user/robot-hand.service
systemctl --user daemon-reload
systemctl --user enable robot-hand.service
systemctl --user start robot-hand.service
```

## En Arduino Mega 2560

1. Conectar por USB al PC
2. Abrir Arduino IDE
3. Cargar mega\_sketch/servos/mega\_servos.ino
4. Seleccionar placa: Arduino Mega 2560
5. Upload

## En STM32U585 (UNO Q)

1. Conectar UNO Q por USB-C al PC
2. Abrir Arduino App Lab
3. Seleccionar target: MCU (STM32U585)
4. Cargar stm32\_sketch/bridge/bridge.ino
5. Upload

## Errores y Soluciones

### Error 1: UNO Q no detectado por USB

**Síntoma:** Placa encendida pero lsusb no muestra dispositivo.

**Causa:** Cable USB sin datos (power-only).

**Solución:** Usar cable USB-C a USB-C con datos.

### Error 2: Permisos USB (udev)

**Síntoma:** App Lab detecta pero no puede hacer provisioning.

**Causa:** /dev/bus/usb/002/00X pertenece a root:root.

**Solución:** Regla udev para VID 2341 / PID 0078:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="2341", ATTR{idProduct}=="0078",  
MODE="0666"
```

Archivo: /etc/udev/rules.d/99-arduino-uno-q.rules

### **Error 3: /dev/ttyACM0 Permission denied**

**Causa:** Usuario en grupo dip, no dialout.

**Solución:** `sudo usermod -a -G dialout $USER`

### **Error 4: pip “externally-managed-environment”**

**Síntoma:** `pip3 install` falla en Debian 13.

**Solución:** Usar `pip3 install --break-system-packages <paquete>`

### **Error 5: Cámara bloqueada en HTTP**

**Síntoma:** `getUserMedia()` no pide permisos.

**Causa:** Navegadores solo permiten cámara en HTTPS o localhost.

**Solución:** Añadir certificado SSL autofirmado al servidor Flask.

### **Error 6: Servidor Flask muere al cerrar SSH**

**Causa:** Proceso background se mata al terminar la sesión SSH.

**Solución:** Usar servicio `systemd (robot-hand.service)` con `linger` habilitado.

### **Error 7: Dedos demasiado sensibles**

**Síntoma:** Ángulos de servos saltan de 0 a 100% al mínimo movimiento.

**Causa:** `FINGER_SCALE` demasiado bajo (0.55).

**Solución:** Usar escala 1:1 (`180 - flexion`), deadband 5°, EMA  $\alpha=0.3$ .

### **Error 8: Thumb tracking no responde**

**Síntoma:** El pulgar no rastrea bien, se queda clavado.

**Causa 1:** Fórmula de abducción invertida.

**Causa 2:** Depende de la orientación de la mano respecto a la cámara.

**Solución:** Usar  $(95 - abduction) \times 2.25$  con debug logging. Pendiente calibración final.

# Estado del Proyecto

## Completado (2026-05-06)

- ☒ Documentación técnica del proyecto
- ☒ Conexión y provisioning del UNO Q
- ☒ Reglas udev para permisos USB
- ☒ Acceso SSH al Qualcomm Linux
- ☒ Instalación de dependencias Python
- ☒ Servidor Flask HTTPS con WebSocket
- ☒ Interfaz web con MediaPipe.js + tracking de mano
- ☒ Motor de cinemática inversa (IK)
- ☒ Control de movimiento suave (EMA + deadband + interpolación)
- ☒ Sketch STM32U585 bridge con control LED matrix
- ☒ Sketch Arduino Mega con interpolación de servos
- ☒ Servicio systemd para auto-arranque Flask
- ☒ Configuración WiFi dual (Xiaomi + Feophone)
- ☒ Soporte SSL/TLS con certificado autofirmado

## Pendiente (para 2026-05-07)

- ☐ Cableado físico UNO Q ↔ Mega (3 cables dupont)
- ☐ Identificar mapeo de 5 servos a pines del Mega
- ☐ Subir sketch STM32 bridge via App Lab (target MCU)
- ☐

- ☐ Subir sketch Mega servos via Arduino IDE
- ☐ Prueba de comunicación manual (M0 90 → servo se mueve)
- ☐ Activar tracking real → servos copian mano
- ☐ Calibrar sensibilidad y ángulos por dedo
- ☐ Calibrar pulgar con mano real
- ☐ Generar PDF final de documentación

## Árbol del Proyecto

```
BrazoRobotico/
├── docs/
│   ├── brazo-robotico.md          # Documentación técnica (este
│   └── errores-soluciones.md      # Bitácora de errores
│       └── brazo-robotico.pdf    # PDF generado
├── stm32_sketch/
│   └── bridge/
│       └── bridge.ino            # Puente UART + LED Matrix
(STM32U585)
├── mega_sketch/
│   └── servos/
│       └── mega_servos.ino       # Control servos con
interpolación (Mega 2560)
├── qualcomm/
│   ├── server.py                 # Flask HTTPS + WebSocket + IK
│   ├── ik_engine.py             # Motor de cinemática inversa
│   └── wifi_ap_setup.sh          # Configuración WiFi AP
"RobotHand"
├── requirements.txt              # Dependencias Python
├── cert.pem                     # Certificado SSL
├── key.pem                      # Clave privada SSL
├── static/
│   └── index.html                # Interfaz web + MediaPipe.js
└── README.md
```

---

*Documento generado el 6 de mayo de 2026. Proyecto Brazo Robótico — Arduino UNO Q.*